Lecture 13: Storage II — File Systems & Inodes

Session 16 · Wed Oct 21, 2026

Contents

Recap: Lecture 12 . . . 1
Outline . . . 1
File System Layers . . . 2
Inodes . . . 2
Run It: stat Reads the Inode . . . 2
Directory Structure . . . 2
Run It: Reading a Directory . . . 3
Looking Up a Path . . . 3
Hard Links vs. Symbolic Links . . . 4
Run It: Links and Link Counts . . . 4
Removing a File Is unlink . . . 4
Run It: A File With No Name . . . 5
Creating a File: The Full Path . . . 5
Where the Bytes Live: Blocks . . . 5
File Descriptors Across fork() . . . 5
Run It: One Position, Two Processes . . . 6
Who Shares a File Position? . . . 6
AI Systems Connection . . . 6
The Lecture in One Slide . . . 7
Practice . . . 7
Answers to Practice . . . 7
Reading . . . 7

Recap: Lecture 12

Lecture 12. A program names an open file with a **file descriptor**, a small integer that indexes the process's descriptor table. Each `open` call makes an **open file table entry**, which holds the file position. `read` and `write` copy bytes and move that position, and `stat` reads a file's size and type without reading its bytes.

Today. The third level of that picture is the file itself. On the disk, a file is an **inode**, a record that holds everything about the file except its name. Names live in directories. Today is about how the kernel gets from a name to an inode, and what happens to the open file table entry when a process calls `fork`.

Outline

What you will learn today:

1. What an inode holds, and why the file's name is not in it
2. What a directory holds, and how the kernel looks up a path
3. Hard links and symbolic links, and what the link count counts (Lab 6, Part A, Problem 5)
4. What `rm` really does, and why a deleted file can still be read
5. What the kernel does when `open` creates a new file
6. How a file's bytes are found on the disk

7. Why a parent and a child share one file position after `fork` (Lab 6, Part A, Problem 2)
8. How a training job saves a checkpoint so that a crash cannot corrupt it

File System Layers

A **file system** is the part of the kernel that stores files on a disk. It keeps two maps: from names to files, and from each file's byte offsets to the disk blocks that hold those bytes.

Layer	What it does
your program	calls `open`, `read`, `write`, `stat`
VFS (Virtual File System)	one set of system calls for every kind of file system
a file system: ext4, xfs, APFS, …	turns names into inodes and offsets into blocks
block layer and driver	reads and writes numbered 4096-byte blocks on the device

Linux machines usually use **ext4**. Macs use **APFS**. Because of the VFS layer, the same `open` and `read` calls work on both, and on USB drives and network file systems too.

Inodes

An **inode** (index node) is the file system's record for one file. Every file has exactly one inode, and the file system numbers them. The inode holds:

- the file's type (regular file, directory, symbolic link, …)
- the permission bits, such as `rw-r--r--`
- the owner and the group
- the size in bytes
- the **link count**, the number of directory entries that name this inode
- three timestamps: last read, last change to the bytes, last change to the inode itself
- where the file's bytes are on the disk

The inode does **not** hold the file's name. A name is stored in a directory, next to the inode number it refers to. An inode number is unique only inside one file system; a USB drive has its own inode 12.

Run It: stat Reads the Inode

The `stat` command prints fields of a file's inode, and never reads the file's bytes. The `-c` format picks the fields: `%i` is the inode number, `%h` the link count, `%s` the size, `%b` the blocks, `%F` the type and `%A` the permissions.

```
$ echo hello > notes.txt
$ stat -c 'inode %i, links %h, size %s, blocks %b, %F, %A' notes.txt   # Linux
inode 22153684, links 1, size 6, blocks 8, regular file, -rw-rw-r--
$ ls -i notes.txt
22153684 notes.txt
```

The inode number is the same number `ls -i` prints. `links 1` means one directory entry names this inode. `blocks 8` counts 512-byte units, so the 6-byte file takes one 4096-byte block on the disk.

Directory Structure

A **directory** is a file whose contents are a list of entries. Each entry holds a name and an inode number, and nothing else:

```
Directory demo/  (inode 22153683)
  .           22153683      this directory itself
  ..          22153654      the parent directory
  notes.txt   22153684
  sub         22153686      a directory is also just an inode
```

`opendir` and `readdir` read the entries one at a time:

```c
DIR *d = opendir(".");
struct dirent *e;
while ((e = readdir(d)) != NULL)              // NULL: no more entries
    printf("%10llu  %s\n", (unsigned long long)e->d_ino, e->d_name);
closedir(d);
```

Because a name is only an entry, `mv notes.txt old.txt` in the same file system rewrites one entry and leaves the inode and the bytes alone. `cp notes.txt copy.txt` makes a new inode and copies the bytes into it.

Run It: Reading a Directory

`01-myls.c` is the loop on the previous slide. Here it lists `demo/`, and then the root directory `/`:

```
$ ./01-myls                          # Linux, ext4
  22153685  link.txt
  22153684  same.txt
  22153684  notes.txt
  22153683  .
  22153686  sub
  22153654  ..
$ ./01-myls / | grep '  \.'
         2  .
         2  ..
```

The entries come out in the order the directory stores them, not in alphabetical order. That is why `ls` sorts them before printing. `.` and `..` are real entries. In `/`, both are inode 2, because the root directory is its own parent. Two names, `notes.txt` and `same.txt`, hold the same inode number. That is a hard link, three slides from now.

Looking Up a Path

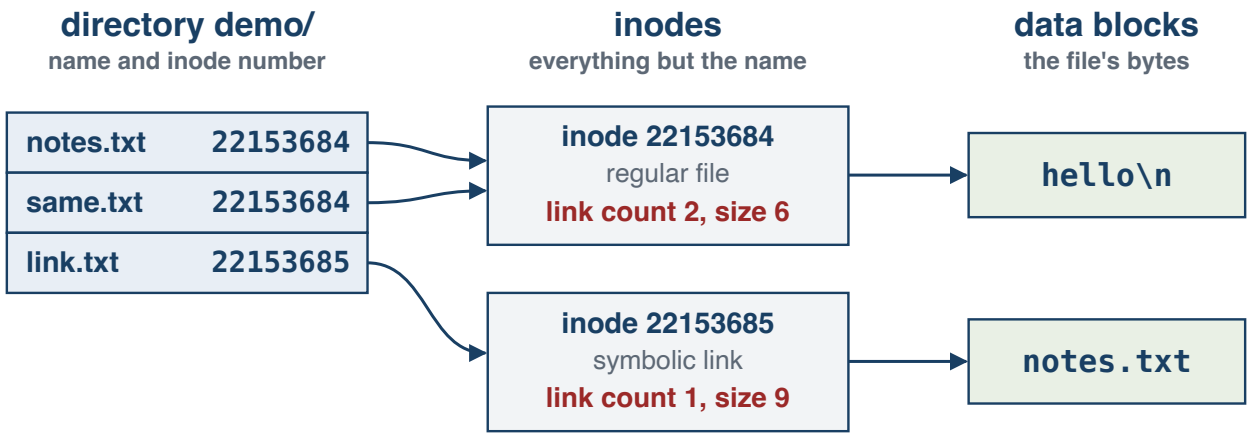
To open `/usr/include/stdio.h`, the kernel looks up one name at a time:

1. Start at the root directory, inode 2.
2. Read inode 2's entries, find `usr`, and get inode 4980737.
3. Read inode 4980737's entries, find `include`, and get inode 4980740.
4. Read inode 4980740's entries, find `stdio.h`, and get inode 5000071.

```
$ stat -c '%i %n' / /usr /usr/include /usr/include/stdio.h     # Linux
2 /
4980737 /usr
4980740 /usr/include
5000071 /usr/include/stdio.h
```

A path that does not start with `/` starts at the process's current directory instead. At every step the process needs **execute** permission on the directory, which for a directory means permission to look names up in it. A long path costs one lookup per name. The kernel keeps recently used names in RAM, in the **dentry cache**, so most lookups never touch the disk.

Hard Links vs. Symbolic Links



notes.txt and same.txt are two names for one inode, so the link count is 2.
link.txt has its own inode. Its data is the name notes.txt, looked up again.

- **Hard link** (`ln notes.txt same.txt`): a second directory entry with the same inode number. Both names are equally real, and the link count is 2.
- **Symbolic link** (`ln -s notes.txt link.txt`): a new inode of type symbolic link whose data is the path `notes.txt`. Opening it makes the kernel look up that path. If the name `notes.txt` is gone, the link is broken.

Run It: Links and Link Counts

The directory from the diagram. The first column of `ls -li` is the inode number, and the column after the permissions is the link count (`-g -o` leave out the owner and group):

```
$ echo hello > notes.txt; ln notes.txt same.txt; ln -s notes.txt link.txt
$ mkdir sub; ls -li -g -o                        # Linux, ext4
22153685 lrwxrwxrwx 1    9 Oct 21 14:08 link.txt -> notes.txt
22153684 -rw-rw-r-- 2    6 Oct 21 14:08 notes.txt
22153684 -rw-rw-r-- 2    6 Oct 21 14:08 same.txt
22153686 drwxrwxr-x 2 4096 Oct 21 14:08 sub
$ ln sub sub2
ln: sub: hard link not allowed for directory
$ mv notes.txt renamed.txt; cat same.txt; cat link.txt
hello
cat: link.txt: No such file or directory
```

`link.txt` is 9 bytes because it stores the 9 characters of `notes.txt`. The new directory `sub` already has link count 2: its entry in `demo/`, and its own `.` entry. After `mv`, the name `notes.txt` no longer exists, so the symbolic link is broken, but the hard link still reaches the inode.

Removing a File Is unlink

There is no system call that deletes a file. `rm` calls `unlink(path)`, which removes **one directory entry** and lowers the inode's link count by one. The kernel frees the inode and its blocks only when both are true:

- the link count is 0, so no directory entry names the inode, and
- no process has the file open.

So `rm` on one of two hard links only removes a name. The other name still works, and the data stays. And a file that a process still has open keeps its blocks after its last name is removed, until the last `close`.

Run It: A File With No Name

`02-unlink.c` creates `temp.txt`, writes 11 bytes, removes the file's only name with `unlink`, and then reads the bytes back through the descriptor:

```
$ ./02-unlink                                    # Linux and macOS
before unlink: link count 1
after unlink:  the name temp.txt is gone
after unlink:  link count 0, size 11 bytes
read 11 bytes through the descriptor: still here
```

The inode still exists with link count 0, because the descriptor keeps it open. Programs use this for temporary files that must disappear even if the program crashes. It is also why deleting a huge log file sometimes frees no disk space: a running server still has the file open, and the space comes back only when the server closes it or exits.

Creating a File: The Full Path

```c
int fd = open("demo/new.txt", O_WRONLY | O_CREAT | O_EXCL, 0644);
```

When `new.txt` does not exist yet, the kernel:

1. Looks up `demo`, the directory that will hold the new name.
2. Checks that the process has write and execute permission on `demo`.
3. Allocates a free inode and sets its type, permissions (0644), owner, size 0, link count 1 and timestamps.
4. Adds the entry `new.txt` with the new inode number to `demo`.
5. Makes an open file table entry with position 0, and returns the lowest unused descriptor.

With `O_EXCL`, if `new.txt` already exists, `open` fails with `EEXIST` and nothing changes. Creating a file writes to the directory, not to the file, which is why it needs write permission on the directory.

Where the Bytes Live: Blocks

The file system divides the disk into **blocks** of 4096 bytes and gives each file whole blocks:

```
$ head -c 5000 /dev/zero > five-k.bin                    # 5000 zero bytes
$ stat -c '%s bytes, %b blocks of %B bytes' notes.txt five-k.bin   # Linux
6 bytes, 8 blocks of 512 bytes          # one 4096-byte block
5000 bytes, 16 blocks of 512 bytes      # two 4096-byte blocks
```

The inode records which blocks hold the file. The classic Unix layout, used by ext2 and ext3, stores 12 block numbers in the inode for the first 48 KB. For a larger file, one more inode field names an **indirect block**, a block that holds 1024 more block numbers (4 MB more). A double indirect block reaches 4 GB, and a triple indirect block 4 TB. ext4 instead stores **extents**: "this file's next N blocks start at disk block B". A file written in one piece usually needs only one extent.

File Descriptors Across fork()

`fork` makes a copy of the running process (Lecture 17). The child gets a copy of the parent's descriptor table, but **not** a copy of the open file table entries. Each copied descriptor points to the same entry as the parent's, so the two processes share one file position:

```c
int fd = open("data.txt", O_RDONLY);      // one open, before fork
char buf[10];

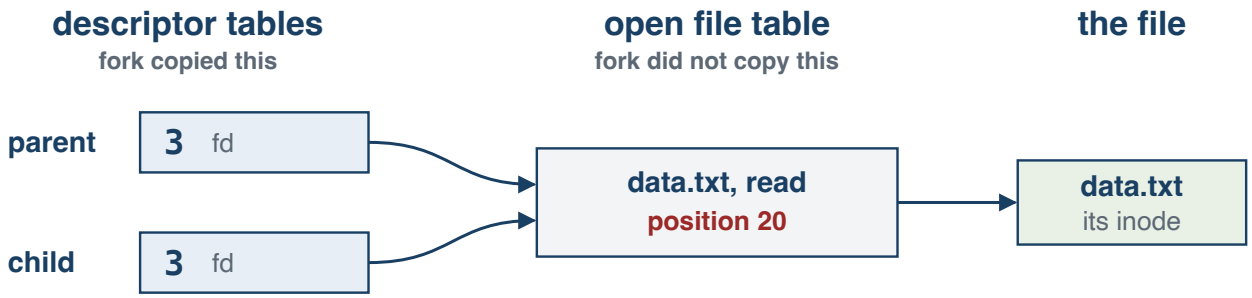
if (fork() == 0) {
    read(fd, buf, 10);     // child: suppose it reads first, bytes 0 to 9
} else {
    read(fd, buf, 10);     // parent: then this reads bytes 10 to 19
}
```

If each process called `open` itself after `fork`, there would be two entries with two positions, and both would read bytes 0 to 9.

Run It: One Position, Two Processes

`03-forkpos.c` is the previous slide's program, with one change: the parent calls `waitpid` first, so the child always reads first. `data.txt` holds the 20 bytes `0123456789abcdefghij`:

```
$ ./03-forkpos                                  # Linux and macOS
child  read 0123456789, position now 10
parent read abcdefghij, position now 20
```



Two descriptors in two processes, one entry, so one position for both.

Who Shares a File Position?

How the two descriptors were made	Entries	Position
two `open` calls	two	separate
one `open`, then `fork`	one	shared
one `open`, then `dup2` (Lecture 18)	one	shared

The shell's `2>&1` (Lab 5, Level 2) uses `dup2`, so standard output and standard error share one position. Two `open` calls do not:

```
$ ls -d / /nope > out.txt 2>&1; cat out.txt             # one entry
ls: cannot access '/nope': No such file or directory
/
$ ls -d / /nope > out.txt 2> out.txt; cat out.txt       # two entries
/
: cannot access '/nope': No such file or directory
```

In the second run, both positions start at 0, so `/` and its newline overwrite the `ls` at the start of the error message.

AI Systems Connection

A training job runs for days and saves the model's weights to a checkpoint file every few thousand steps, so that a crash does not lose the whole run. If the job overwrites `ckpt.pt` in place and crashes halfway through, the file holds half new and half old weights, and the last good checkpoint is gone.

The safe way uses today's ideas:

1. Write the new checkpoint to a new file, `ckpt.pt.tmp`, and call `fsync` to wait until its bytes are on the disk.
2. Call `rename("ckpt.pt.tmp", "ckpt.pt")`. It changes one directory entry to hold the new inode number, in one step. Any process that opens `ckpt.pt` gets either the complete old file or the complete new one.

3. The old inode's link count drops to 0. A process that was already reading the old checkpoint keeps reading it until it closes it.

`rename` is a single step only inside one file system. Between two file systems, `mv` has to copy the bytes.

The Lecture in One Slide

- An inode holds everything about a file except its name: type, permissions, owner, size, link count, timestamps and block locations.
- A directory is a list of (name, inode number) entries. The kernel looks up a path one name at a time, starting at `/` or the current directory.
- A hard link is another entry with the same inode number. A symbolic link is a separate inode that holds a path.
- `rm` calls `unlink`, which removes one name. The inode is freed when its link count is 0 and no process has it open.
- `fork` copies the descriptor table but shares the open file table entries, so parent and child share one file position.
- `rename` replaces a directory entry in one step, which makes it the safe way to replace a file.

Practice

1. `notes.txt` has inode 500 and link count 1. You run `ln notes.txt b.txt` and then `rm notes.txt`. What are `b.txt`'s inode number and link count? Are the bytes still on the disk?
2. Why does the kernel refuse a hard link to a directory?
3. You run `ln -s /usr/include/stdio.h link.txt`. What size does `ls -l link.txt` show, and why?
4. A program opens `log.txt` with `O_WRONLY | O_CREAT | O_TRUNC` and then calls `fork`. The child writes `child\n`; the parent waits for the child and then writes `parent\n`. What does `log.txt` hold? What would it hold if each process had called `open` itself after `fork`?
5. The disk is full. You `rm` a 10 GB log file, but `df` shows no space freed. Why, and what will free it?

Answers to Practice

1. Inode 500, link count 1. `ln` raised the count to 2 and `rm` lowered it back to 1. The bytes stay, because `b.txt` still names the inode.
2. Every directory must have exactly one parent, the one its `..` entry names. A second hard link would give it two, and could make a loop, so a program that walks the tree, such as `find` or `rm -r`, could visit the same directory forever.
3. 20 bytes. A symbolic link stores the path it was given, and `/usr/include/stdio.h` has 20 characters.
4. `child\nparent\n`. Both processes share one position, so the parent writes at byte 6, after the child's bytes. With two `open` calls there are two entries. The parent's `open` cuts the file to 0 bytes (`O_TRUNC`), and its write starts at its own position 0, so the file holds only `parent\n`.
5. A running process still has the file open, so the inode and its blocks are not freed. The space comes back when that process closes the file or exits.

Reading

- CS:APP §10.6–10.11 (File metadata, directories, sharing files, I/O redirection, standard I/O)
- OSTEP Chapter 40: File System Implementation
- `man 2 unlink`, `man 2 link`, `man 2 rename`, `man 3 readdir`
- The sample code in `sample code/lecture13/`